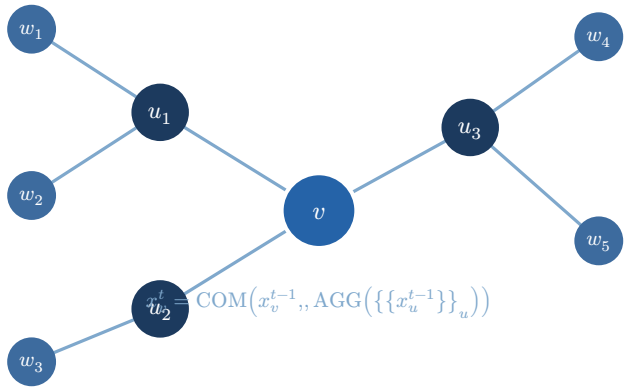
Recurrent Graph Neural Networks

Logical Characterizations via Modal Logic



Kevin Kunkel & Thomas Mohr

Summer Semester 2026

Universität Leipzig — Seminar: Graph Neural Networks
Supervisor: Prof. Carsten Lutz

Outline

Talk Overview

Tom	**2.1 — Graph Neural Networks** Message passing, standard vs. recurrent, floats vs. reals	5 min
Kevin	**2.2 — Logics** GML, GMSC, ω-GML, and their hierarchy	5 min
Tom	**3 — Connecting GNNs and Logics via Automata** Distributed automata, main theorems, GNN[F] vs. GNN[R]	20 min
Kevin	**4 — Characterizing GNNs over MSO Properties** What is MSO? The collapse theorem	20 min
Tom Kevin	**Conclusion**	10 min

> **Paper:** Ahvonen, Heiman, Kuusisto, Lutz — NeurIPS 2024 [1]

GNNs

Each node v maintains a **feature vector** updated in rounds:

$$x_v^{(t)} = \varphi\Big(x_v^{(t-1)},, \oplus_{u \in \mathcal{N}(v)}\, \psi\Big(x_v^{(t-1)}, x_u^{(t-1)}\Big)\Big)$$

- **AGG** — aggregate neighbor feature vectors (e.g. sum, max)
- **COM** — combine own vector with aggregated result (e.g. MLP, GRU)

Standard GNN: run for a **fixed** N rounds, then read out.
N is a hyperparameter — must be chosen in advance.

Standard vs. Recurrent GNNs

Standard GNN

- Runs exactly N rounds
- N fixed before computation
- After N rounds: accept or reject
- **Problem:** must know N upfront

Recurrent GNN

- No fixed round limit
- Runs until a node reaches an **accepting** state
- A special **DONE** vector signals termination
- The GNN decides **itself** when it is done

Remark What happens when computation can take **arbitrarily long**? E.g. a loop that iterates until some condition is met? — We need an adaptive stopping criterion.

Recurrent GNNs — Formal Model

Definition: Recurrent GNN A recurrent GNN$[\mathbb{R}]$ over (Π, d) is a tuple $\mathcal{G} = \big(\mathbb{R}^d, \pi, \delta, F\big)$:

- **init** $\pi : \mathcal{P}(\Pi) \to \mathbb{R}^d$, **transition** $\delta(x, y) = \mathrm{COM}(x, \mathrm{AGG}(y))$, **accept** $F \subseteq \mathbb{R}^d$

$\mathcal{G}$ **accepts** (G, v) iff $x_v^t \in F$ for **some** $t \in \mathbb{N}$.

Replace $\mathbb{R}$ with floating-point $\mathbb{F} \longrightarrow \mathrm{GNN}[\mathbb{F}]$.

Example **Reachability of label p:** Use $C = A = 1, b = 0$ (1-dimensional). Round 0: state is 1 if $p \in \lambda(v)$, else 0. Round t: state is 1 if node or any neighbor is 1. Propagates through the whole graph — regardless of size.

GNN[F] vs. GNN[R] — Why It Matters

In practice GNNs use **floats** ($\mathbb{F}$). Theory usually assumes **reals** ($\mathbb{R}$).

> **The float problem:** Floating-point addition is **not associative**: $(a + b) + c \neq a + (b + c)$
>
> In a 2-decimal system: $1 + (-1) + 0.01 = 0.01$, but $(1 + 0.01) + (-1) = 1.0 + (-1) = 0$ because 1.01 is not representable and gets rounded.

Consequence (Proposition 2.3): For every float system $\mathbb{F}$ there exists $k \in \mathbb{N}$ such that for all multisets M of floats:

$$\mathrm{SUM}_{\mathbb{F}}(M) = \mathrm{SUM}_{\mathbb{F}}(M|_k)$$

Past k copies of a value, additional copies make **no difference** — floats can't count beyond a fixed bound.

Logics

Graded Modal Logic (GML) ~~Kevin~~

GML is propositional logic extended with **counting modalities**:

$$\varphi ::= \top \texttt{ mid } p \texttt{ mid } \neg\varphi \texttt{ mid } \varphi \wedge \varphi \texttt{ mid } \Diamond_{\geq k} \varphi$$

The key formula $\Diamond_{\geq k} \varphi$ means: **"at least k out-neighbors satisfy φ"**

Example On **graphs:**

- $\Diamond_{\geq 1} p$ — "there is a neighbor with label p"
- $\Diamond_{\geq 3} (q \wedge \Diamond_{\geq 2} r)$ — "at least 3 neighbors have q and each has ≥2 neighbors with r"

Classic result [2]: constant-iteration GNNs ≡ GML (relative to FO-definable properties)

Graded Modal Substitution Calculus (GMSC)

GMSC extends GML with **recursive rules** — a program Λ consists of:

```
X(0) :- φ    // base case: starting formula (GML)
X    :- ψ    // iteration: rule that may use X recursively
```

The n-th unfolding X^n: substitute X in ψ by X^{n-1}, starting from $X^0 = \varphi$.

Λ accepts (G, v) if $G, v \vDash X^n$ for **some** n.

Example **Reachability of p:** `X(0) :- p` `X :- ◇X`
$X^i = \Diamond \cdots \Diamond p$ (exactly i diamonds) = reachability in i steps

ω-GML and the Logic Hierarchy

ω-GML adds **infinite disjunctions** of GML formulas:

$$\varphi ::= \psi \quad | \quad \bigcup_{\psi \in \Psi} \psi \quad (\Psi \text{ a countable set of GML formulas})$$

Since GNN[R] with real numbers can distinguish **arbitrarily many** values, it needs this infinite expressive power.

Expressive power:

$$\text{GML} \subset \text{GMSC} \subset \omega\text{-GML}$$

Example **On strings:**
FO ≡ star-free regular languages
MSO ≡ all regular languages

ω-GML and the Logic Hierarchy

On graphs:
FO can say: “every node has a neighbor”
MSO can say: “the graph is bipartite”, “there exists a path from a to b”

GNNs and Logics via Automata

Definition: Counting Message-Passing Automaton (CMPA) A CMPA (Q, q_0, δ, F) runs on graphs:

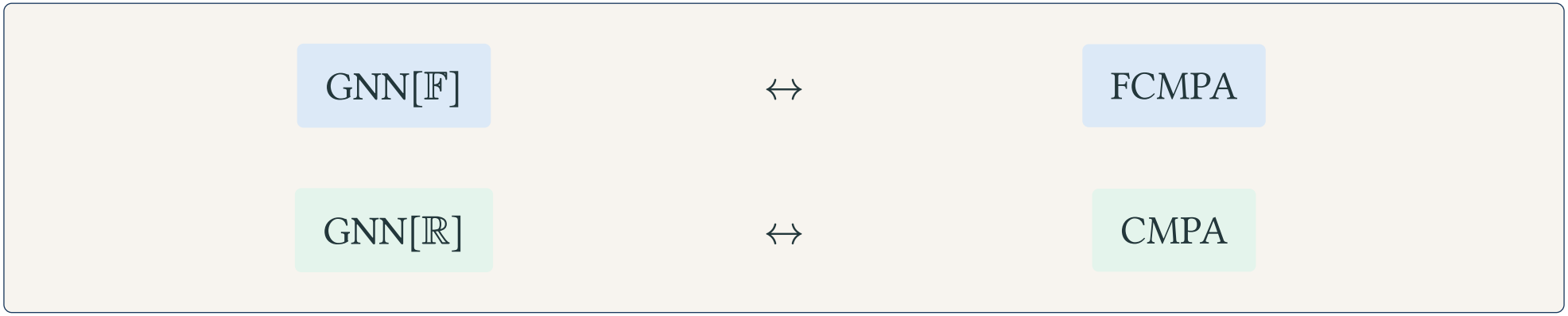
- Each node starts in state q_0
- Each node updates its state based on its **own state** and the **multiset** of neighbor states: $q_v^t = \delta(q_v^{t-1}, \{\{q_u^{t-1} \mid (v,u) \in E\}\})$
- Node v **accepts** iff $q_v^t \in F$ for some t

FCMPA — finite state set Q **CMPA** — countably infinite Q

CMPAs are like GNNs but with **discrete** states. This makes them easier to connect with logic.

The Three-Way Correspondence

The key insight: GNNs, automata, and logics form a **tight triangle**:



- GNN[$\mathbb{F}$] $\leftrightarrow$ FCMPA because floats have bounded counting (Prop. 2.3)
- GNN[$\mathbb{R}$] $\leftrightarrow$ CMPA because reals can distinguish any multiset size exactly
- Both automaton classes then connect to their respective logic

Why GNN[R] is Strictly More Powerful

The counting argument:

GNN[$\mathbb{R}$] can express: **"The number of out-neighbors is a prime."**

For any $U \subseteq \mathbb{N}$ — even **undecidable** ones — GNN[$\mathbb{R}$] can check whether the neighbor count lies in U.

GNN[$\mathbb{F}$] with bound k cannot distinguish graphs with k vs. $k+1$ neighbors.

So GNN[$\mathbb{F}$] **cannot** express primality of degree for arbitrary degree.

Absolute result: GNN[$\mathbb{F}$] $<$ GNN[$\mathbb{R}$].
But the extra power of reals lies in properties no one ever needs in practice — which is the surprising result coming up in Section 4.

Main Theorems

Theorem: GNN[F] ≡ GMSC — Theorem 3.2 [1] The following have the **same expressive power** — absolutely, no background logic:

$$\mathrm{GNN}[\mathbb{F}] \quad \equiv \quad \text{R-simple AC-GNN}[\mathbb{F}] \quad \equiv \quad \mathrm{GMSC} \quad \equiv \quad \mathrm{FCMPA}$$

Theorem: GNN[R] ≡ ω-GML — Theorem 3.4 [1]

$$\mathrm{GNN}[\mathbb{R}] \quad \equiv \quad \mathrm{CMPA} \quad \equiv \quad \omega\text{-GML}$$

ω-GML can define **undecidable** graph properties — so GNN[ℝ] is very powerful.

Both results are **absolute** — no relativization to a background logic required.

GNNs over MSO Properties

What is MSO? ~~Kevin~~

MSO (Monadic Second-Order Logic) extends FO with **set quantification**:

- FO: quantify over **elements** ($\exists x$, $\forall x$)
- MSO: additionally quantify over **sets of elements** ($\exists X$, $\forall X$)

On graphs:

FO can say:
"every node has a neighbor"

MSO can say:
"the graph is bipartite"
"there exists a path from a to b"
many global structural properties

On strings:

FO $\equiv$ star-free regular languages

MSO $\equiv$ all regular languages

(a clean, well-known correspondence)

MSO is in a sense the **natural** logic for graphs — it captures almost all properties one cares about in practice.

MSO — What It Can Express

MSO-expressible graph properties include:

- Connectivity
- k-colorability
- Hamiltonicity
- Bipartiteness
- Planarity (for fixed genus)
- Existence of a path between two labeled nodes
- Many more natural structural properties

Remark MSO is **not** the limit of what GNNs can do. Section 3 showed GNN[$\mathbb{R}$] can go far beyond MSO. But MSO covers everything that matters in practice.

The MSO Collapse Theorem

Theorem: MSO Collapse — Theorem 4.3 [1] For any property $\mathcal{P}$ expressible in MSO:

$$\mathcal{P} \text{ expressible as } \mathrm{GNN}[\mathbb{R}] \quad \Leftrightarrow \quad \mathcal{P} \text{ expressible as } \mathrm{GNN}[\mathbb{F}]$$

Both are further captured by GMSC. This also holds for constant-iteration GNNs.

Absolutely: $\mathrm{GNN}[\mathbb{R}] > \mathrm{GNN}[\mathbb{F}]$

Extra power: "degree is prime", undecidable properties.

Relative to MSO: $\mathrm{GNN}[\mathbb{R}] = \mathrm{GNN}[\mathbb{F}]$

For all practically relevant properties, floats are **just as good**.

Why the Gap Vanishes — Intuition

The proof goes via **Parity Tree Automata (PTAs)** — which characterize MSO on tree-structured graphs (Janin–Walukiewicz theorem):

MSO property $\mathcal{P} \to$ PTA $A \to k$-prefix decorations $\to$ GMSC program Λ

Key insight: The extra power of GNN[$\mathbb{R}$] lies entirely **outside** of MSO.
If $\mathcal{P}$ is in MSO and expressible by GNN[$\mathbb{R}$], one can always find a GMSC program — and hence a GNN[$\mathbb{F}$] — for it.

Practical consequence: Theoretical analyses with $\mathbb{R}$ are safe — for MSO properties the results transfer to floats. If a GNN doesn't learn an MSO-definable property, the problem is in training or architecture, not float precision.

Summary

Results at a Glance ~~Kevin~~ ~~Tom~~

Setting	GNN[F]	GNN[R]	Automaton
Absolute	$\equiv$ GMSC	$\equiv$ ω-GML	F: FCMPA, R: CMPA
Relative to MSO	$\equiv$ GMSC	$\equiv$ GMSC (!!)	$\equiv$ FCMPA

Key takeaway:

- Absolutely: $\mathrm{GNN}[\mathbb{F}] < \mathrm{GNN}[\mathbb{R}]$ — reals can express undecidable properties
- Relative to MSO: $\mathrm{GNN}[\mathbb{F}] \equiv \mathrm{GNN}[\mathbb{R}]$ — **theory and practice converge**

Conclusion & Open Questions

What we showed:

- Exact logical characterizations: $\mathrm{GNN}[\mathbb{F}] \equiv \mathrm{GMSC}$, $\mathrm{GNN}[\mathbb{R}] \equiv \omega\text{-GML}$
- Both results are **absolute** — no background logic needed
- The real–float gap **vanishes** for all MSO-definable properties
- Floats lose nothing vs.
  reals for any property one would use in practice

Open questions:

- **Termination:** How does a recurrent GNN learn **when** to stop? Not addressed.
- **Global readout:** How does the characterization change with a global pooling layer?
- **Complexity:** Is expressibility in GMSC decidable?
- **Attention architectures:** Where do GAT / Transformer fit in this framework?

References

References

[1] V. Ahvonen, D. Heiman, A. Kuusisto, and C. Lutz, "Logical Characterizations of Recurrent Graph Neural Networks with Reals and Floats," *arXiv preprint arXiv:2405.14606*, 2024.

[2] P. Barceló, E. V. Kostylev, M. Monet, J. Pérez, J. Reutter, and J. P. Silva, "The logical expressiveness of graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2020.